SBOM과제 연구노트 (2025년 7월)

생성 일시	@2025년 9월 19일 오전 11:39

연구주제 #3 - SBOM 변환

연구 목표

- SBOM 형식 데이터를 타 소프트웨어에서 사용 가능한 데이터로 연산하여 제공
- SBOM 표준 (SPDX, CycloneDX) 간 상호 변환

4차년도 개발 내용

SBOM 연산/변환기술 성능 고도화 및 프레임워크 개발

→ 산출물: SBOM 데이터 분석 도구

연구 수행 내용

SBOM Harmonization Plugfest 2024 분석보고서 내용 요약

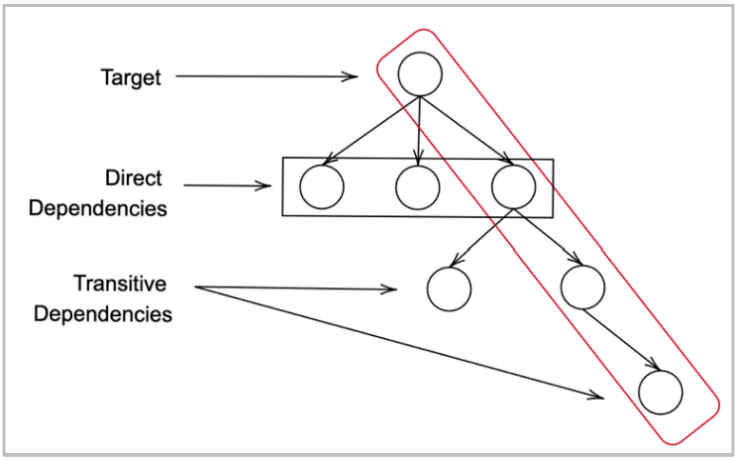

Figure 1: Example Dependency Tree Illustrating Target Software Dependencies, Depth, and Breadth

- SBOM depth: Length of longest path (red box)

- SBOM breadth: Max. number of components at any given distance from target software (black box)

Summary

- 21 Plugfest participants → 243 SBOMs submitted
- Analysis order:
  - JSON SBOMs in CycloneDX format (most common)
  - JSON SBOMs in SPDX format
  - XML and YML SBOMs (Hatbom과는 거리 존재)

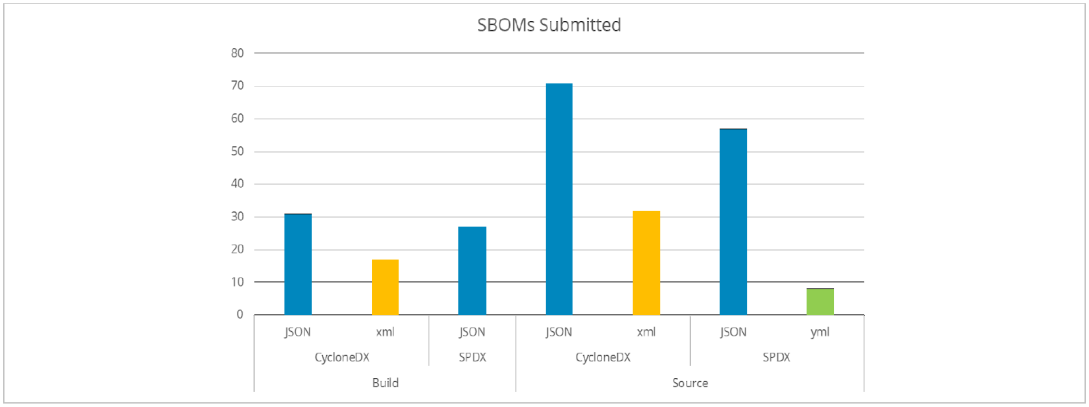

Figure 3: Distribution of Submitted SBOMs by Standard and Format

SBOM Depth Analysis

- Calculating depth values:
  - SPDX v2.3: “DEPENDS ON”, “DEPENDENCY_OF” relationships
    - Relationship types were limited to “METAFILE_OF”, “CONTAINS”, “DESCRIBES”, “DEPENDS_ON”, “DEPENDENCY_OF”, “OTHER”
    - Many other relationship types are allowed within the SPDX standard → **Hatbom도 더 다양한 방법으로 의존성 표현 고려 필요**
  - CycloneDX: “DEPENDENCIES” elements

Findings

- jq (C) & OpenCV (C++) were targets with lowest commonality in components **(Hatbom이 제출한 SBOM 대상)**
- Lack of normalization for component names and versions is one cause of variance
  - Version flexibility leads to ambiguity, since SBOMs include single version for each dependency
  - PURL specification has new syntax for version ranges
    - Version range specifier: https://github.com/package-url/purl-spec/blob/main/VERSION-RANGE-SPEC.rst
- Other factors for SBOM variance:
  - Varying conventions for specifying dependencies
  - Different definitions/interpretations of a "dependency"
  - Diversity of SBOM use cases (different specifications for different uses)
- Some SBOMs captured dependency information from external locations:
  - Artifact repositories referenced by dependencies
  - Contents of platform toolchain libraries
- Variance in degree of inclusion of minimum required elements in SBOMs

Recommendations (Hatbom에 해당되는 내용만)

SBOM Minimum Elements

- **SBOM Type**: For which lifecycle phase was this SBOM generated? (e.g., Source, Build)
  - Hatbom은 Source SBOM을 생성
- **Component Version String**: Version info. normalization is needed + should follow semantic versioning formats that allow flexibility in reporting version ranges
  - 위처럼 PURL의 version range specifier 사용 고려
- **Component Supplier Name**:

> For open source software components, which do not have a traditional supplier, a direct reference or link to the project repository should be provided.

- **Component Cryptographic Hash**: What is being hashed to create the cryptographic hash?
  - 각 component의 대한 hash값이 어떻게 생성되었는지에 대한 description 포함하면 좋을 듯 하다
- Component License: Provide licensing information, or note that **such info is not known / not included**
  - Hatbom은 후자에 해당될 듯 하다

SBOM Harmonization

- Terminology:
  - “supplier”: primary supplier
  - “manufacturer”: secondary supplier
- Component Inclusion Reason: standardize annotation of each component with reason for inclusion (examples below)
  - The component was included in a build manifest (e.g., pom.xml).
  - The component was used by a package manager during build.
  - The component was used by a platform toolchain during build.
  - The file was processed by a compiler.
  - The file was read to determine the license for a component.

Future Research

- Vulnerability analysis & how well SBOMs support vulnerability management function
  - VEX documents were not considered for the current Plugfest
- Analysis of SBOM structure with respect to dependencies

    - Compare multiple SBOMs that represent identical target softwares using dependency trees → highlight similarities and differences across SBOM structures
    - Hatbom DIFF 연산 개선 방안으로 고려
- SBOM evolution: how to track and implement changes to SBOMs over time

- Hatbom 한정 개선방안: 소프트웨어 외부 의존성 (modules, packages) 정보 프로세싱 필요